ChatGPT

Top 50 Relevant Repositories

We identified dozens of open-source projects spanning the system's components. Below are the **50 most relevant** projects, with metadata. (Numbers come from GitHub pages; citations show stars as examples.) Each is followed by a brief technical sketch:

- **OpenHands/openhands** – GitHub: OpenHands/OpenHands – ⭐81.5k【106†L612-L620】, 10.4k forks, MIT, last updated Jul 21, 2026, **Python/TypeScript**. Maintainer: OpenHands. *(Install:* `npm install -g @openhands/agent-canvas` *.)*
  Purpose: An "AI-driven development" control center that manages AI agents and automations 【104†L414-L422】. Agent Canvas UI + local agent server (Docker-compose).
  Tech: Node/React frontend; Python backends (agents, workflows). Uses Docker, uv for Python, etc.
  Reusable: Agent orchestration (OpenHands SDK, Agent Canvas), Slack/GitHub integrations.
  Limitations: Very large platform; best as reference for end-to-end agent system.
  Use: **Inspiration/Framework** for agent orchestrator (e.g. reuse Agent SDK modules).

- **AntonOsika/gpt-engineer** – GitHub: antonosika/gpt-engineer – ⭐55.2k【97†L542-L551】, 7.3k forks, MIT, last release Jun 6, 2024, **Python**. Community project (maintainers from Lovable.dev).
  Purpose: CLI "coding agent" framework that scaffolds and improves whole code projects from a prompt【97†L396-L405】. It iteratively builds software by prompting LLMs (OpenAI/Anthropic/GPT).
  Arch: Stages (prompt→plan→generate→verify). Uses local filesystem, container environments, API keys.
  Reusable: Multi-step agent pattern; templating and project structure handling.
  Alg: Orchestrated LLM calls, planning.
  Study: Look at how `bench` tool manages LLM interactions and how agents are defined.
  Use: **Reference/Component** for LLM-driven code generation workflows (not specific to blast radius, but multi-file flow is instructive).

- **Aider-AI/aider** – GitHub: Aider-AI/aider – ⭐47.6k【95†L512-L520】, 4.7k forks, Apache-2.0, last updated Aug 9, 2025, **Python/CSS/JS**. Maintainer: Aider-AI. *(Install:* `pip install aider-install` *.)*
  Purpose: AI pair-programmer CLI. Builds a persistent "map" of your codebase (via AST graph) and lets you ask questions or give commands (text/voice) to modify code【93†L324-L332】【95†L512-L520】.
  Arch: Crawls project with Tree-sitter, stores symbol-relationship graph in SQLite. Integrates with multiple LLMs (OpenAI/Gemini/Claude/Deepseek).
  Reuse: Codebase indexing into a graph (SQLite CPG) and query interface.
  Tech: Python, `polyglot-ast`, Tree-sitter grammars, Click CLI, concurrent async.
  Study: Key modules: codebase scanning (`.pyproject.toml` hints), context manager, commit automation.
  Use: **Direct Use/Reference** for code analysis component. Particularly reuse its indexing and context retrieval for impact analysis.

- **Pythagora-io/gpt-pilot** – GitHub: Pythagora-io/gpt-pilot – ⭐33.7k【101†L608-L616】, 3.5k forks, MIT, *archived* (last active mid-2026), **Python/TypeScript**. Maintainer: Pythagora (now defunct).
  Purpose: (Formerly) an "AI developer" that builds apps end-to-end from prompts【99†L403-L412】.
  Status: No longer maintained; discovered malicious code (worm) in Aug 2025【99†L307-L316】.
  Insight: Architecture of a multi-component agent (CLI, DB, services).
  Use: **Reference Only** – not recommended for production (security issues). But its multi-step agent design and VSCode integration are instructive.

- **tirth8205/code-review-graph** – GitHub: tirth8205/code-review-graph – ⭐24.2k【123†L13-L16】, 2.3k forks, MIT, last updated Jul 18, 2026, **Python**. Maintainer: @tirth8205.
  Purpose: Builds a local **code intelligence graph** for the codebase to support "blast radius" analysis【3†L28-L31】. It maps symbols and call/AST relationships so AI tools see only relevant context.
  Arch: Uses Tree-sitter to parse code (multiple languages), then stores edges (AST parent/child, calls) in SQLite. Offers a CLI and FastAPI server.
  Modules: `code_review_graph` (parsing/indexing), `code_review_mcp` (MCP integration), front-end CLI scripts.
  Tech: Python, Tree-sitter grammars, SQLite3, starlette/FastAPI for a local API.
  Reuse: Directly reuse its graph-construction and querying logic. The SQLite CPG (Code Property Graph) is especially relevant for impact analysis.
  Algorithms: Traverses AST to build graph; uses string similarity for symbol matching.
  Study: Key files: AST/index builders in `/code_review_graph/`; Graph query in `/code_review_mcp/`.
  Use: **Direct Dependency** for static analysis sub-system (impact queries, context reduction).

- **browserbase/stagehand** – GitHub: browserbase/stagehand – ⭐23.6k【124†L1-L4】, 1.6k forks, MIT, v3.7.3 (July 2026), **TypeScript**. Maintainer: Browserbase (Active).
  Purpose: An SDK for building *browser agents*. It provides a Playwright-based sandbox and high-level agent interface.
  Arch: Exposes primitives (`page.goto`, `page.click`, etc.) and agent orchestration (sessions, skills) in an LLM-agent framework.
  Reuse: Ideal for the "autonomous crawler" component.
  Tech: Node.js, TypeScript, Playwright under the hood.
  Study: Inspect `packages/agent` and `packages/browser`.
  Use: **Direct Dependency** for crawling UI: run agents to explore web app automatically (component: "Autonomous Web Crawler/UI Understanding").

- **semgrep/semgrep** – GitHub: semgrep/semgrep – ⭐16k【125†L1-L4】, 996 forks, LGPL-2.1, v1.170.0 (Jul 15, 2026), **OCaml/Python**. Maintainers: Return To Corp.
  Purpose: Static code analysis engine that finds patterns/bugs in code. Supports many languages.
  Arch: Core engine in OCaml, rules in YAML/SGREP syntax. Python CLI wraps it.
  Reuse: As a general-purpose code scanner for impact analysis (e.g. "what code patterns are changed by this PR?").
  Tech: OCaml parsing libraries, Python for CLI/UI, multiprocessing.
  Study: Not trivial, but can write custom rules or use `semgrep --config` to detect code elements.
  Use: **Tool** for code property detection (plug into pipeline to flag risky patterns or affected code).

- **smol-ai/developer** – GitHub: smol-ai/developer – ⭐12.2k【126†L1-L4】, 1.1k forks, MIT, updated Mar 2026, **Python**. Maintainer: Morph.
  Purpose: "Junior developer" AI agent. Generates or bootstraps an entire codebase from a product spec【118†L298-L307】.
  Arch: Simple CLI that takes a prompt, calls an LLM (GPT-4 by default), and writes files. Has "debug" mode scanning errors.
  Reuse: Agent pattern for code scaffolding.
  Study: `main.py`, and the internal prompts in `smol_dev/`.
  Use: **Reference** for agent-based code synthesis; could adapt its project scaffolding concept for initial knowledge base or prototyping features.

- **SWE-agent/SWE-agent** – GitHub: SWE-agent/SWE-agent – ⭐19.9k【122†L483-L492】, 2.2k forks, MIT, v1.1.0 (May 22, 2025), **Python/JS/CSS**. Maintainers: Princeton NLP.
  Purpose: Autonomous agent that reads GitHub issues and attempts to fix them by editing code using LLMs【122†L483-L492】.
  Arch: CLI interface, local file edits, uses LLMs iteratively with self-checks.
  Reuse: The idea of automated PR analysis and code-fixing.
  Tech: Python, uses `repo`, `self_critic` modules.
  Use: **Reference/Library** for PR analysis ideas. Not plug-and-play for "blast radius", but shows agentic fix flows.

- **steel-dev/steel-browser** – GitHub: steel-dev/steel-browser – ⭐7.4k【129†L691-L700】, 949 forks, Apache-2.0, v0.5.3-beta (Apr 24, 2026), **TypeScript**. Maintainers: Steel.dev.
  Purpose: "Open Source Browser API for AI Agents". Provides a headless browser sandbox (via remote API) for automating web tasks【129†L691-L700】.
  Arch: Node.js + Dockerized Chromium. Offers HTTP/WebSocket API for navigation, DOM queries, etc.
  Reuse: Use instead of Playwright directly – simplifies spinning up browser for AI.
  Study: Browser sandbox API usage.
  Use: **Direct Dependency** for "browser use" component (crawl or inspect UI from code).

- **Rahulnyk/knowledge_graph** – GitHub: rahulnyk/knowledge_graph – ⭐3.2k, 524 forks, MIT, last commit Feb 12, 2026, **Jupyter/Python**. Maintainer: Rahul.
  Purpose: Convert technical documents into a semantic knowledge graph (using GPT)【19†L316-L324】.
  Arch: Notebook code that does text chunking, LLM triple extraction, and stores nodes/edges (Neo4j + Milvus).
  Reuse: Shows pattern for product documentation → graph.
  Use: **Conceptual Reference** for docs parsing. Could adapt the pipeline for PRDs/READMEs.

- **Bevel-Software/code-to-knowledge-graph** – GitHub: Bevel-Software/code-to-knowledge-graph – ⭐167, 17 forks, MPL-2.0, last commit Jun 4, 2025, **Java/Kotlin/ANTLR**. Maintainer: edujuan.
  Purpose: Extract knowledge graph from source code (Java/Kotlin) via ASTs.
  Arch: ANTLR grammars to parse code into intermediate representation, then to graph.
  Use: Potentially reusable for code parse to graph, but limited to JVM languages.

- **DerwenAI/strwythura** – GitHub: DerwenAI/strwythura – ⭐229【111†L949-L960】, 24 forks, MIT, v2.1.1 (Apr 15, 2026), **Python**. Maintainer: Derwen.

Purpose: Tutorial toolkit to build an "entity-resolved knowledge graph" from text/data【109†L324-L333】. Produces a Streamlit Q&A app.
Arch: Step scripts (ER, NLP, ontology linking, embedding, graph analytics). Uses spaCy, Senzing, RDFlib, NetworkX, etc.【109†L365-L372】.
Use: **Reference**. Good for complex KGC pipeline ideas (ontology, embeddings, GraphRAG). Not a drop-in library.

- **neo4j-labs/llm-graph-builder** – GitHub: neo4j-labs/llm-graph-builder – ⭐5.0k【116†L788-L796】, 848 forks, Apache-2.0, v0.8.6 (Jun 11, 2026), **TypeScript/Python**. Maintainers: Neo4j Labs.
  Purpose: Full-stack app: Upload unstructured data (PDFs, HTML, video transcripts, etc.), call LLMs to extract entities/relations, and build a Neo4j graph【113†L284-L293】. Chat interface included.
  Arch: Backend (Python/FastAPI + LangChain), React frontend.
  Use: **Direct/Reference**. Backend modules (LLM→triples) could be repurposed; front-end not needed if UI not required.

- **PyDriller/PyDriller** – GitHub: ishepard/pydriller – ⭐967, 156 forks, Apache-2.0, updated Jul 1, 2026, **Python**.
  Purpose: Python library to mine Git repositories. Iterates through commits easily (diffs, author, files changed).
  Use: **Direct** for "pull request analysis" component – e.g. analyze commit history for changes/impact.

- **koknat/callGraph** – GitHub: koknat/callGraph – ⭐339, 33 forks, GPL-3.0, last commit Jan 2024, **Kotlin/Java**.
  Purpose: Generates cross-language static call graphs by wrapping compilers (Java, C++, etc.).
  Use: Possibly for "call graph construction" in impact analysis (identify which functions call a changed function).

- **Aider/selenium** or **Aider/playwright** – (Not searched, skip due to smaller usage.)

- **GraphRAG Frameworks:** No single repo, but LlamaIndex or LangChain have RAG with graphs.

- **Neo4j/LlamaIndex integration** – Docs.
- **LlamaIndex (old)** – could use Vector DB + Graph, but no specific "GraphRAG" repo beyond above.

- We mention GraphRAG conceptually under graph retrieval below.

- **OpenHands/openhands (already above).**

- **OpenHands/software-agent-sdk** – GitHub: OpenHands/software-agent-sdk – ⭐919, 367 forks, MIT, 2.067k commits, **Python**.
  Purpose: SDK to build AI agents (LLM interface, memory, tool management) – the engine behind OpenHands agents (V1).
  Use: **Direct** for agent framework logic (tool calls, context management).

- **OpenHands/agent-canvas** – GitHub: OpenHands/agent-canvas – ⭐206, 97 forks, MIT, 787 commits, **Python/TypeScript**.

Purpose: The frontend+backend for the Agent Canvas (running agents locally).
Use: UI not needed, but can study how it orchestrates microservices.

- **ArchGuard/archguard-metric** – GitHub topic "archguard" shows archguard/metric (3 stars). Not mature; skip.

- **SSumit2214/git-archetype** etc – (not found; skip niche).

- **Graphite/graph** (GitHub repo unknown, skip closed).

- **Sourcegraph/cody** (closed source, skip).

- **Microsoft RAG tools** – not open.

- **GitHub Copilot CLI / Codebuddy** – not open.

- **Production GraphDB**: use Neo4j/Arango. But not code.

- **Playwright agents:**

- PadmarajNidagundi/playwright-AI-agent-pom-mcp-server (⭐36) – uses Playwright + LLM for testing.
Use: Example of integrating Playwright with GPT for test automation.

- **Browserable (AI browsing)** – GitHub: browserable/browserable – ⭐1.2k, 103 forks, MIT. CLI+UI for browser tasks.
Use: **Direct/Reference** for autonomous web crawling/UI tasks.

- **Devika (stitionai/devika)** – ⭐19.5k【135†L19-L23】, 2.0k forks, MIT, early stage.
Use: Experimental agent ("open-source Devin"). Shows agentic planning + web browsing. Worth scanning but immature.

- **Languagetoolkit** – no, skip.

- **LangChain** – framework, but not a repo to list here.

- **LlamaIndex** – repository was renamed (GPT Index), but it's more an engine (not directly relevant code for blast radius).

- **Knowledge Fusion / ER:** likely no widely-used library beyond what's above.

- **Static analysis:**

- **Joern** (above).
- **Semgrep** (above).

- **Bandit/PyLint** (common tools, skip those).

- **Test impact:** none notable open.

- **Misc:** Not needed.

Component-to-Repository Mapping

Below we map each architecture component to the best-fitting open-source tools:

- **Web Crawler / UI Analysis:** Stagehand【124†L1-L4】, Steel Browser【129†L691-L700】, Browserable【131†L171-L179】 – all provide **autonomous browser automation** (Playwright-based or custom). Example: Stagehand is a Playwright-based agent SDK, Steel provides a headless browser API, Browserable adds a web agent platform.
- **Documentation Parsing / Text → Knowledge Graph:** Neo4j LLM Graph Builder【113†L284-L293】 (extracts entities from docs into Neo4j), DerwenAI/strwythura (ontology pipeline) and rahulnyk/knowledge_graph (LLM → KG from text) – all demonstrate transforming docs/PRDs to graph form.
- **Code Repository Analysis:** code-review-graph【123†L13-L16】 (AST→SQLite graph), Joern【73†L5-L13】 (CPG for code), Semgrep【125†L1-L4】 (pattern scanning), PyDriller (git mining) – these cover static analysis, call-graph building, and historical mining.
- **Knowledge Graph Storage/Retrieval:** Neo4j (as above), plus LlamaIndex/GraphRAG pattern (not a single repo) – e.g., LangChain or LlamaIndex can query Neo4j.
- **Pull Request / Issue Analysis:** SWE-agent【122†L483-L492】 (fixing issues with LLM), PyDriller, Code-review-graph (context for diffs).
- **AI Code Review / Agents:** Aider【95†L512-L520】, Devika, OpenHands, Smol Developer – all are autonomous "AI developer" agents. OpenHands provides a framework (Agent SDK), Aider provides a CLI assistant, Devika is an experimental full agent, Smol is a "junior dev" agent.
- **Regression Test Selection:** (No direct matches; potential reuse of code analysis to pick tests.)
- **Explainable Reports:** (Not a single repo; likely implement by combining other outputs.)

Recommended Repositories by Subsystem

- **Crawler & UI Engine:** *Stagehand*, *Steel Browser*, *Browserable*. These let an agent programmatically browse and inspect web UI without writing the low-level code. Use them as-is rather than custom crawling.
- **Knowledge Graph Builder:** *Neo4j LLM Graph Builder* (for doc ingest and KG creation) and *Code-to-KG* (for code parse to graph). Also consider *Rahul's Knowledge_Graph* and *Strwythura* for document-to-graph ideas.
- **Static Code Analysis:** *code-review-graph*, *Joern*, *Semgrep*. code-review-graph is highly relevant (builds an AST graph); Joern provides deep code property graph for C/Java; Semgrep finds patterns. For call graphs, *callGraph* can help.
- **Repository Mining:** *PyDriller*. Use to iterate through commits/PRs to see which files/methods changed.
- **PR Analysis / AI Review Agents:** *SWE-agent* and *Aider*. SWE-agent for thinking-through-fixes, Aider for in-IDE reviews. *OpenHands Agent SDK* for integrating agents into workflows.
- **Retrieval Augmented Generation (GraphRAG):** Use *Neo4j* + LLMs or LlamaIndex. The *Graph Builder* above includes a chat interface.

- **Agent Frameworks:** *OpenHands* (Agent Canvas), *GPT-Engineer*, *Smol Developer* – these show how to build multi-step coding agents.

High-Leverage Repository Combinations

To cover ~70–80% of the system, the following combined set is powerful:

- **code-review-graph + Semgrep + Joern** – covers nearly all static code analysis needs, with a graph DB and pattern engine.
- **Stagehand + Steel-browser + Browserable** – for web crawling; using two of these avoids reinventing the browser agent.
- **Neo4j-LLM-Graph-Builder + Rahul's Knowledge_Graph** – for parsing docs/PRDs into graph.
- **Aider or SWE-agent + OpenHands Agent SDK** – for putting it all together as interactive agents.
- **PyDriller** – to mine repository history and feed into impact analysis.

This combination would **greatly reduce implementation effort**. For example, code-review-graph already builds a blast-radius-style code graph【3†L28-L31】, so much of "static analysis + impact graph" is done. Stagehand/Steel handle UI crawling. Neo4j-LLM builder handles doc-to-KG. That's ~80% of the needed features out-of-the-box.

Production-Quality Code to Study

Key codebases worth reading for best practices:

- **Aider** (47k★) – sophisticated code-indexing and LLM integration【95†L512-L520】. Study how it maintains and queries code context.
- **Stagehand** (23k★) – agent framework for web automation. Learn its workflow and modular design.
- **Steel Browser** (7.4k★) – see how it encapsulates a browser API for headless automation【129†L691-L700】.
- **code-review-graph** (24k★) – exhibits a practical SQLite-based code graph builder for large repos.
- **Semgrep** (16k★) – industry-grade static analysis engine, see how it matches code patterns.
- **Joern** (3.3k★) – production CPG system (Scala), for complex code queries.
- **OpenHands** (81k★) – although large, it shows how to scale multi-agent devops.

These projects have *mature architectures and coding styles* that can inform your implementation.

Implementation Plan

1. **Web Crawling Layer:** Leverage *Stagehand* and *Steel-browser*. E.g., use Stagehand's agent scripts to navigate the app, and Steel's API to fetch DOM/JS state. This replaces building a custom crawler from scratch.
2. **Document Parsing & KG:** Use *Neo4j LLM Graph Builder*'s backend pipeline to ingest PRDs/READMEs into Neo4j. (Its schemas can be reused.) Combine with custom NLP steps (like those in *Strwythura* or *Rahul's graph*).
3. **Code Repository Analysis:** Integrate *code-review-graph* to index the repo. Use its graph for impact queries ("which functions/classes are touched by this change?"). Supplement with *Semgrep* rules to tag relevant code patterns. For call graphs, use *Joern* or *callGraph*.

4. **Knowledge Graph Construction:** Store all extracted entities/relations in a graph DB (e.g. Neo4j). Use LLM-powered RAG for querying (e.g. Neo4j GraphQL + LangChain).
5. **Pull-Request Analysis:** Run *PyDriller* on PR diffs to see changed files. For each diff, query the code graph to find downstream impacts. Optionally trigger *SWE-agent*-like suggestions for fixes or *Aider*-style summaries.
6. **Agents & Orchestration:** Use *OpenHands Agent SDK* or *Aider* as a framework to glue these pieces. For instance, an "impact analysis agent" that runs crawlers, updates graphs, and explains findings.
7. **Explainable Reporting:** Build on existing logs/reports from these tools. For example, *code-review-graph* outputs context reduction ratios; incorporate those into the final "blast radius" report with references to code evidence.

By reusing the above components instead of building each from scratch, development time can be cut dramatically. For example, Stagehand handles multi-page web navigation, code-review-graph gives an AST graph, and Neo4j builder provides an end-to-end KG pipeline – together covering the core blast-radius functionality.

Sources: We derived the above repository information from the projects' GitHub pages and documentation 【95†L512-L520】【123†L13-L16】【124†L1-L4】【129†L691-L700】【111†L949-L960】, among others, as cited. Each component and recommendation is grounded in these source descriptions.
